



ASISTENTE EXPERTO CON GEMINI, RAG Y AGENTES

El objetivo de este proyecto es construir un agente de inteligencia artificial capaz de responder preguntas sobre un tema definido por el alumno, utilizando su propia base de conocimiento vectorial. El agente aplicará técnicas de Retrieval-Augmented Generation (RAG) con Google Gemini como modelo de lenguaje y embeddings, incluirá memoria de conversación y podrá interactuar mediante texto desde un notebook.

Este proyecto integra todos los conceptos trabajados en las dos primeras semanas del módulo: LLMs, ingeniería de prompts, RAG, LangGraph, agentes con memoria y pipelines ML + GenAI.

STACK TECNOLÓGICO

Todos los componentes del proyecto deben construirse con las siguientes tecnologías:

- LLM y Embeddings: Google Gemini
- Base de conocimiento vectorial: ChromaDB
- Framework de agente: LangGraph (con LangChain como base)
- Entorno de trabajo: Jupyter Notebook o Google Colab
- (Opcional) Interfaz web: Streamlit

OBJETIVOS DE APRENDIZAJE

Al completar este proyecto, el alumno será capaz de:

- Crear y gestionar una base de conocimiento vectorial con ChromaDB usando Gemini Embeddings
- Implementar un agente en LangGraph que combine RAG y el LLM de Gemini.
- Diseñar prompts efectivos (system prompt documentado y justificado) y mantener coherencia conversacional.
- Integrar memoria de conversación en el agente para mantener contexto entre preguntas.
- Interactuar con el agente desde un notebook y, opcionalmente, desplegarlo en Streamlit.

ALCANCE DEL PROYECTO

1. MVP — Funcionalidades Mínimas

- Agente experto en un dominio definido por el alumno.
- Base de conocimiento vectorial creada con un mínimo de 3 documentos o equivalente a ~20 páginas de texto (PDF, CSV o dataset propio), indexados en ChromaDB con Gemini Embeddings.
- Respuesta a preguntas usando RAG + Gemini como LLM.
- System prompt personalizado y justificado en la documentación.
- Memoria básica de conversación para mantener contexto entre turnos.
- Interacción por texto dentro de un notebook (celda de chat).

2. Opcional — Bonus

- Interfaz web en Streamlit para conversar con el agente.
- Despliegue público de la app en Streamlit Cloud para pruebas externas.



ASISTENTE EXPERTO CON GEMINI, RAG Y AGENTES

INSTRUCCIONES PASO A PASO

Paso 1 — Definir el tema del experto

- Elegir un dominio de conocimiento específico (ej.: finanzas, medicina, documentación técnica, electrónica, moda, gastronomía, derecho, etc.).
- El tema debe permitir recopilar al menos 3 documentos con contenido sustancial.

Paso 2 — Diseñar el system prompt

- Cargar documentos en ChromaDB
- Preprocesar los datos si es necesario: limpieza, segmentación de texto (chunking), eliminación de ruido.
- Verificar que la colección vectorial responde correctamente a consultas de prueba antes de conectar el agente.

Paso 3 — Crear la base de conocimiento

- Definir el rol del agente, su tono, limitaciones y cómo debe comportarse cuando no tiene información suficiente.
- Documentar el prompt en el README junto con una breve justificación de las decisiones de diseño.

Paso 4 — Crear el agente en LangGraph

- Configurar el grafo de LangGraph para recibir preguntas, recuperar contexto de ChromaDB y generar respuestas con Gemini.
- Asegurar que el agente usa correctamente la base de conocimiento vectorial en cada consulta.

Paso 5 — Agregar memoria de conversación

- El agente debe recordar el contexto de preguntas anteriores y mantener coherencia a lo largo de la conversación.
- Incluir al menos un ejemplo en el notebook donde se demuestre que la memoria funciona (pregunta que hace referencia a una respuesta anterior).

Paso 6 — Probar la interacción en el notebook

- Implementar una celda interactiva donde se pueda conversar con el agente y ver respuestas inmediatas.
- Incluir un mínimo de 5 preguntas de ejemplo con sus respuestas documentadas.

Paso 7 (OPCIONAL) — Despliegue en Streamlit

- Crear una interfaz simple para enviar preguntas y recibir respuestas.
- Desplegar la app en Streamlit Cloud para acceso externo.

ENTREGABLES

1 — Notebook funcional (.ipynb) con:

- Base de conocimiento vectorial con ChromaDB + Gemini Embeddings.
- PAgente LangGraph con RAG, Gemini y memoria.
- VCelda de interacción con mínimo 5 ejemplos documentados.



ASISTENTE EXPERTO CON GEMINI, RAG Y AGENTES

2 — README con:

- Descripción del dominio elegido.
- Instrucciones de instalación y ejecución.
- Justificación del system prompt.
- Requisitos (dependencias, API key de Gemini).

3 — Presentación oral de 5-10 minutos con demo en vivo el día de entrega.

4 — (Opcional) URL pública de la app en Streamlit Cloud.

CRITERIOS DE EVALUACIÓN

La evaluación se realiza sobre los siguientes criterios. La interfaz Streamlit es bonus y no penaliza su ausencia

Criterio	Excelente (100%)	Correcto (70%)	Mejorable (<50%)
Base de conocimiento (Chroma + Gemini Embeddings)	Indexación correcta, mínimo 3 docs / ~20 páginas	Funciona pero documentación escasa	Errores o contenido insuficiente
RAG con Gemini (recuperación + generación)	Recuperación precisa, respuestas coherentes con el contexto	RAG funciona pero respuestas poco precisas	RAG no integrado o no funcional
Ingeniería de Prompts	System prompt bien diseñado y justificado en el README	Prompt básico, sin justificación	Sin prompt personalizado
Agente con memoria (LangGraph)	Memoria funcional, coherencia entre turnos de conversación	Memoria básica o parcial	Sin memoria implementada
Calidad general del código y documentación	Código limpio, README claro con instrucciones de ejecución	Código funcional, documentación mínima	Código difícil de seguir, sin documentación
Presentación y demo (5-10 min)	Demo fluida, explicación clara de decisiones técnicas	Demo funciona, explicación básica	Demo con errores o sin presentación
BONUS: Interfaz Streamlit	Interfaz funcional y desplegada	Interfaz local sin despliegue	-



ASISTENTE EXPERTO CON GEMINI, RAG Y AGENTES

CONSEJOS Y BUENAS PRÁCTICAS

- Guarda tu API key de Gemini en una variable de entorno o en un archivo .env, nunca en el código.
- Empieza con un dataset pequeño para probar el pipeline completo antes de escalar la base de conocimiento.
- Itera sobre el system prompt: pequeños cambios en el prompt pueden mejorar mucho la calidad de las respuestas.
- Usa el modo de tracing de LangGraph para depurar el flujo del agente si algo no funciona como esperas.
- Documenta las decisiones técnicas mientras desarrollas, no al final — te ahorrará tiempo en la presentación.